

Simulado — Programação Orientada a Objetos

20 questões de múltipla escolha · 0,5 ponto cada · Total: 10 pontos

Prof. Rodrigo Martins Pagliares · Gabarito ao final

GRASP

Questão 1

Em um sistema de reservas de hostel, a classe **Hostel** mantém uma lista de reservas. Qual princípio GRASP justifica atribuir à classe **Hostel** a responsabilidade de calcular a taxa de ocupação?

- (a) Creator, pois Hostel cria objetos do tipo Reserva.
- (b) Controller, pois Hostel recebe eventos externos do sistema.
- (c) Information Expert, pois Hostel possui as informações sobre reservas necessárias para o cálculo.
- (d) Pure Fabrication, pois a operação de cálculo não existe no domínio real.
- (e) Indirection, pois Hostel atua como intermediário entre Reserva e o resultado.

Questão 2

Considerando o princípio GRASP **Creator**, qual das seguintes afirmativas justifica corretamente que a classe **Venda** deve criar instâncias de **LinhaDeltemDeVenda**?

- (a) Venda é uma classe abstrata que define o contrato para LinhaDeltemDeVenda.
- (b) Venda agrega objetos LinhaDeltemDeVenda e possui os dados necessários para inicializá-los.
- (c) Venda herda de LinhaDeltemDeVenda, adquirindo a capacidade de criá-la.
- (d) LinhaDeltemDeVenda implementa a interface de Venda, tornando a criação natural.
- (e) Venda atua como Controller, recebendo eventos de sistema que disparam a criação.

Questão 3

Uma classe **GerenciadorDeReservas** foi criada sem correspondente no domínio real, com o único propósito de manter a coesão das demais classes. Qual princípio GRASP essa decisão exemplifica?

- (a) Information Expert
- (b) Low Coupling
- (c) Controller
- (d) Pure Fabrication
- (e) Protected Variations

Questão 4

O princípio GRASP **Low Coupling** é mais diretamente suportado por qual outro princípio?

- (a) Information Expert, pois concentra responsabilidades em uma única classe.
- (b) Indirection, pois introduz um objeto intermediário que desacopla os elementos.
- (c) High Cohesion, pois classes coesas sempre têm baixo acoplamento.
- (d) Creator, pois reduz o número de objetos que precisam criar outros.
- (e) Controller, pois centraliza toda a lógica de comunicação.

Questão 5

Ao projetar um sistema, o desenvolvedor percebe que a lógica de desconto varia conforme o tipo de cliente. Para evitar blocos *if/else* extensos verificando o tipo, qual princípio GRASP deve ser aplicado?

- (a) Protected Variations, encapsulando o ponto de variação com uma interface estável.
- (b) Pure Fabrication, criando uma classe artificial para cada tipo de desconto.
- (c) High Cohesion, garantindo que cada classe trate apenas de um tipo de cliente.
- (d) Polymorphism, atribuindo o comportamento variante a cada subclasse por meio de herança.
- (e) As alternativas d e a estão corretas e se complementam.

Coleções

Questão 6

Assinale a alternativa **INCORRETA** sobre a hierarquia da Java Collections API:

- (a) As interfaces Set, List e Queue herdam da interface Collection.
- (b) A interface Map herda de Collection, pois também é uma estrutura de dados.
- (c) ArrayList implementa a interface List.
- (d) LinkedList implementa tanto List quanto Queue.
- (e) HashSet implementa a interface Set.

Questão 7

Qual é a diferença entre uma coleção **ordered** e uma coleção **sorted** na Java Collections API?

- (a) Ordered usa índices numéricos; sorted usa chaves alfanuméricas.
- (b) Ordered e sorted são sinônimos na API — ambos garantem a mesma ordem.
- (c) Ordered garante uma ordem de iteração previsível (ex: inserção); sorted ordena por regra natural ou Comparator.
- (d) Sorted usa ordenação por inserção; ordered usa ordenação por valor natural.
- (e) Ordered é exclusivo de listas; sorted é exclusivo de conjuntos.

Questão 8

Um hostel precisa armazenar hóspedes sem permitir duplicatas e com **iteração na ordem de chegada**. Qual implementação é a mais adequada?

- (a) HashSet, pois não permite duplicatas e tem alto desempenho.
- (b) TreeSet, pois mantém os elementos em ordem de chegada.
- (c) ArrayList, pois mantém a ordem de inserção.
- (d) LinkedHashSet, pois mantém a ordem de inserção e não permite duplicatas.
- (e) HashMap, pois associa cada hóspede a um identificador único.

Questão 9

Sobre o método **add()** da interface **Set** em Java, assinale a alternativa correta:

- (a) Lança uma exceção do tipo IllegalArgumentException quando o elemento já existe.
- (b) Substitui o elemento existente pelo novo elemento adicionado.
- (c) Retorna false quando o elemento já existe no conjunto, sem lançar exceção.
- (d) Retorna true sempre, independente de o elemento já existir.
- (e) Adiciona o elemento em uma posição aleatória, podendo gerar duplicatas temporárias.

Questão 10

Qual é a principal **vantagem** de usar ArrayList<Reserva> em vez de um array Reserva[]?

- (a) ArrayList permite armazenar tipos primitivos diretamente, ao contrário do array.
- (b) ArrayList tem acesso por índice mais rápido do que o array.
- (c) ArrayList tem tamanho dinâmico, eliminando a necessidade de definir um tamanho fixo na declaração.
- (d) ArrayList não permite duplicatas, garantindo integridade dos dados.
- (e) ArrayList ordena automaticamente os elementos pela ordem natural.

Exceções

Questão 11

Sobre a hierarquia de exceções em Java, assinale a afirmativa **INCORRETA**:

- (a) RuntimeException é subclasse de Exception.
- (b) Exceções verificadas (checked) devem ser declaradas com throws ou capturadas com catch.
- (c) Exceções não-verificadas (unchecked) são subclasses de RuntimeException ou Error.

- (d) Exceções verificadas não precisam ser especificadas na assinatura dos métodos nem tratadas.
- (e) O bloco finally é executado independentemente de uma exceção ter sido lançada.

Questão 12

No padrão de criação de exceções customizadas visto em aula (classe **ExcecaoHospedeNaoEncontrado**), por que o construtor deve chamar `super(mensagem)`?

- (a) Para invocar o construtor de `Object` e alocar memória para o objeto.
- (b) Para inicializar o atributo `message` herdado da classe `java.lang.Exception`.
- (c) Para registrar a exceção no log do sistema automaticamente.
- (d) Para garantir que a exceção seja classificada como `checked` pelo compilador.
- (e) Para que o método `getMessage()` seja sobrescrito corretamente na subclasse.

Questão 13

Considere o trecho: `try { return 1; } finally { return 2; }`. Qual valor é retornado?

- (a) 1, pois o `return` no `try` tem precedência.
- (b) 2, pois o bloco `finally` sempre executa e sobrescreve o `return` do `try`.
- (c) O código não compila, pois não é permitido usar `return` dentro de `finally`.
- (d) Lança uma exceção `RuntimeException` por conflito de retornos.
- (e) O comportamento é indefinido e depende da JVM utilizada.

Diagrama de Sequência do Sistema

Questão 14

Qual é a principal diferença entre um **Diagrama de Sequência do Sistema (DSS)** e um **Diagrama de Sequência (DS)**?

- (a) O DSS usa notação UML 1.x; o DS usa notação UML 2.x.
- (b) O DSS mostra o sistema como caixa preta com ator e `:Sistema`; o DS mostra objetos internos colaborando.
- (c) O DSS pertence à fase de design; o DS pertence à fase de análise.
- (d) O DSS e o DS são nomes diferentes para o mesmo artefato UML.
- (e) O DSS só pode ter um ator; o DS pode ter múltiplos atores.

Questão 15

Ao nomear operações de sistema em um DSS, qual das alternativas representa o **melhor** nome para a operação em que o caixa informa o identificador de um produto?

- (a) `escanear(idProduto, qtd)`, pois descreve a ação física realizada.
- (b) `setIdProduto(idProduto)`, pois segue o padrão `JavaBeans`.
- (c) `processarProduto(idProduto, qtd)`, pois é um verbo de ação.
- (d) `entrarItem(idProduto, qtd)`, pois usa verbo genérico sem sugerir implementação.
- (e) `handleProduto(idProduto)`, pois segue convenção de nomenclatura de eventos.

Modelos de Análise e Design OO

Questão 16

Sobre o **Modelo de Domínio** no Processo Unificado, assinale a alternativa **INCORRETA**:

- (a) O Modelo de Domínio mostra classes conceituais do mundo real, não classes de software.
- (b) Normalmente não é necessária a inclusão de métodos em um Modelo de Domínio.
- (c) Classes conceituais em um Modelo de Domínio normalmente são derivadas de substantivos presentes no domínio.
- (d) O Modelo de Domínio engloba conceitos que descrevem objetos de software e como eles colaboram.
- (e) Usamos os cenários de um caso de uso como entrada para a criação do Modelo de Domínio.

Questão 17

A respeito de análise e design orientados a objetos, assinale a alternativa correta:

- (a) Análise OO tem como ênfase a definição dos objetos de software e como eles colaboram para satisfazer requisitos.
- (b) Design OO tem como ênfase encontrar e descrever os conceitos no domínio do problema.
- (c) Análise OO tem como ênfase encontrar e descrever conceitos no domínio do problema.
- (d) Análise e Design OO são sinônimos e podem ser usados de forma intercambiável.
- (e) Design OO não é necessário quando há um Modelo de Domínio bem definido.

Contrato de Operação

Questão 18

Em um contrato de operação para `entrarItem(idItem, qtd)`, qual das seguintes é uma **pós-condição** redigida corretamente?

- (a) O sistema deve criar uma nova instância de `LinhaDeItemDeVenda`.
- (b) Uma instância de `LinhaDeItemDeVenda` foi criada e associada à Venda corrente.
- (c) O método `entrarItem()` precisa receber `idItem` e `qtd` como parâmetros.
- (d) A Venda corrente deve existir antes de `entrarItem()` ser invocado.
- (e) O sistema verificará se o item existe no catálogo de produtos.

Generics

Questão 19

Selecione a alternativa **INCORRETA** sobre *Generics* em Java:

- (a) Generics não podem ser usados com tipos primitivos como `int` ou `double`.
- (b) Com Generics, a checagem de tipo é feita em tempo de compilação.
- (c) Generics eliminam conversões (castings) desnecessários.
- (d) A definição do tipo genérico é mantida em tempo de execução.
- (e) O operador diamante (`<>`) permite inferência do tipo genérico na instanciação.

Design para Visibilidade

Questão 20

Em um método de **Registradora**, o objeto **Venda** é declarado como variável local dentro do método para calcular o total. Que tipo de visibilidade é essa?

- (a) Visibilidade de atributo, pois `Venda` é referenciada dentro da classe.
- (b) Visibilidade de parâmetro, pois `Venda` é recebida como argumento.
- (c) Visibilidade global, pois `Venda` é acessível em todo o sistema.
- (d) Visibilidade local, pois `Venda` é declarada localmente dentro do método.
- (e) Visibilidade de herança, pois `Registradora` herda de `Venda`.

GABARITO COMENTADO

Q1 (GRASP) → (c)

Information Expert: a responsabilidade é atribuída à classe que TEM a informação. `Hostel` possui a lista de reservas, logo é a especialista para calcular ocupação.

Q2 (GRASP) → (b)

Creator: B deve criar A se B agrega/contém A OU possui os dados para inicializá-la. `Venda` agrega linhas de item e tem os dados necessários.

Q3 (GRASP) → (d)

Pure Fabrication: classe artificial (não existe no domínio real) criada para atribuir responsabilidades de forma a manter alta coesão e baixo acoplamento.

Q4 (GRASP) → (b)

Indirection introduz um intermediário (ex: Controller entre UI e domínio), desacoplando os elementos diretamente — suportando Low Coupling.

Q5 (GRASP) → (d)

Polymorphism (GRASP): variações de comportamento baseadas em tipo devem ser tratadas com polimorfismo (@Override), não com if/else de verificação de tipo.

Q6 (Coleções) → (b)

Map NÃO herda de Collection. Map é uma hierarquia separada na API. Set, List e Queue herdam de Collection, mas Map não.

Q7 (Coleções) → (c)

Ordered = ordem de iteração previsível (como a de inserção, ex: LinkedHashSet). Sorted = ordem definida por regra natural (Comparable) ou Comparator (ex: TreeSet).

Q8 (Coleções) → (d)

LinkedHashSet: é um Set (sem duplicatas) e ordered (mantém a ordem de inserção). TreeSet seria sorted (ordem natural), não de inserção.

Q9 (Coleções) → (c)

Set.add() retorna false silenciosamente se o elemento já existe — não lança exceção. Retorna true apenas quando o elemento é efetivamente inserido.

Q10 (Coleções) → (c)

A principal desvantagem do array é o tamanho fixo na declaração. ArrayList cresce dinamicamente. Arrays têm acesso O(1) similar ao ArrayList.

Q11 (Exceções) → (d)

Exceções verificadas (checked) DEVEM ser declaradas com throws ou capturadas. Afirmar que não precisam ser especificadas é INCORRETO — esse era o gabarito da questão 4.3e da prova2.

Q12 (Exceções) → (b)

super(mensagem) chama o construtor de Exception que inicializa o atributo message (herdado de Throwable). Assim getMessage() retorna a mensagem corretamente.

Q13 (Exceções) → (b)

O bloco finally SEMPRE executa. Quando finally contém um return, ele sobrescreve qualquer return do bloco try. O valor retornado será 2.

Q14 (Diagrama de Sequência do Sistema) → (b)

DSS = análise, sistema como caixa preta (:Sistema). DS = design, objetos internos visíveis mostrando como o sistema implementa as operações.

Q15 (Diagrama de Sequência do Sistema) → (d)

entrarItem() usa verbo genérico. escanear() sugere implementação tecnológica (leitor de código). set/get são padrão de atributos. processarX/handleX são verbos vagos.

Q16 (Modelos de Análise e Design OO) → (d)

INCORRETO: o Modelo de Domínio descreve conceitos do mundo real, NÃO objetos de software. Objetos de software são definidos no Modelo de Projeto (fase de design).

Q17 (Modelos de Análise e Design OO) → (c)

Análise OO = WHAT (conceitos do domínio do problema). Design OO = HOW (objetos de software e colaboração). As alternativas a e b invertem as definições.

Q18 (Contrato de Operação) → (b)

Pós-condições descrevem o estado APÓS a execução, usando voz passada: 'foi criada', 'foi associada'. Alternativa d seria pré-condição. c e e são responsabilidades/pré-condições.

Q19 (Generics) → (d)

Type Erasure: em Java, a informação do tipo genérico é APAGADA em tempo de execução (erased). A checagem ocorre apenas em tempo de compilação.

Q20 (Design para Visibilidade) → (d)

Visibilidade local: o objeto é declarado como variável local dentro do método — temporária, válida apenas durante a execução daquele método.